SuperMarket Express — Study Guide

A map of the whole project: what each part does, how the parts talk to each other, and the reasoning behind the main decisions. Read this first, then open the files in the "Suggested reading order" at the bottom. Every source file also starts with a comment explaining its job.

---

1. What the app does

An internal web app for one grocery store with three kinds of users:

Role	Can do	Screen (URL)
Customer	Self-checkout on their own loyalty account	`/self-serve`
Cashier	Ring up any member's sale, look at stock (read-only)	`/cashier`
Admin	Everything: POS, inventory, order history, loyalty log, analytics, giving people roles	`/admin`

Everyone who signs up is a **customer** with a loyalty account. An admin can also make someone a cashier and/or admin. After signing in, a plain customer goes straight into the shop; a cashier picks Customer or Cashier; an admin picks any of the three (admins can always use the Cashier screen too).

Main features (from the project spec):

1. **Inventory**: products with SKU, barcode, brand, price, cost, category, reorder level, description and photo; stock arrives in **batches** with an expiry date; damaged/spoiled stock is **written off**; expired stock is removed automatically. A taken SKU or a look-alike product is warned about.
2. **POS checkout**: search or scan products, build a basket, pick a member (by phone number, loyalty ID, name or email), pay by Cash or Card (type the amount), print or download the receipt. If the amount is short, loyalty points (₱5 each) can cover the rest; with enough points the whole bill can be paid with points, saved as paid with Loyalty Points.
3. **Loyalty**: earn **1 point per ₱100** paid, redeem at **₱5 per point**, every change logged in an audit trail; members' phone numbers are added here.
4. **Orders** (admin): every sale with its lines, member, payment and points. Each line keeps its price, cost and name from the moment of sale, so editing a product later never changes past orders or past profit.
5. **Analytics** (admin): browse one day, month or year (‹ ›): orders, items sold, gross sales, total cost and net income, a chart, the orders of that period, payment mix and top products.

---

2. The tools (tech stack)

Tool	What it is	Why it's here
React 18	Builds the screens out of small pieces called components	The whole UI
TypeScript	JavaScript with types (`price: number`)	Catches mistakes before running
Vite	Dev server + build tool	`npm run dev` / `npm run build`
React Router	Maps URLs (`/admin`) to pages	`src/App.tsx`
Tailwind CSS v4	Styling with small class names (`px-4`, `text-sm`)	All styling; colours are named in `src/index.css`
Base UI	Unstyled accessible pieces (dialog, dropdown, menu)	Pop-ups and dropdowns work with the keyboard and screen readers
Supabase	Hosted Postgres database + sign-in + file storage	All data, logins, product pictures
zod	Checks form input against rules	`src/lib/validation.ts`
Recharts	Charts	Analytics tab
lucide-react	Icons	Everywhere
jsbarcode	Draws barcodes	`BarcodeImage.tsx`

Commands (run inside `supermarket/`):

```
npm install          # once, to download the libraries
npm run dev          # start the app at http://localhost:3000
npm run typecheck    # check TypeScript without running anything
npm run build        # make the production version (also checks everything compiles)
npm run db:push      # apply new files in supabase/migrations to the live database
```

3. Folder map

```
supermarket/
├─ index.html                 the one HTML page; React fills in <div id="root">
├─ public/logo.svg            the logo (also the browser-tab icon)
├─ public/products/           photos of the actual demo products (free-licensed,
│                             from Wikimedia Commons; CREDITS.md credits each one)
├─ src/
│  ├─ main.tsx                starts React, loads fonts and CSS, wraps the app in providers
│  ├─ App.tsx                 URL → page table, and which pages need which role
│  ├─ index.css               design tokens (colours, corners) + base styles
│  ├─ App.css                 the few rules Tailwind can't do (barcode, receipt printing)
│  │
│  ├─ pages/                  one file per URL
│  │  ├─ LandingPage.tsx      public home page; sign-in pop-up opens on top at /login
│  │  ├─ RolePickerPage.tsx   /app: "How would you like to continue?"
│  │  ├─ SelfServePage.tsx    /self-serve (customer)
│  │  ├─ CashierPage.tsx      /cashier: POS + read-only stock
│  │  └─ AdminPage.tsx        /admin: all tabs, plus the data loading for them
│  │
│  ├─ components/             the building blocks of the pages
│  │  ├─ AppShell.tsx         sidebar layout around every signed-in page (+ Homepage link)
│  │  ├─ AccountMenu.tsx      the account pop-up (sidebar bottom, landing top bar)
│  │  ├─ ProtectedRoute.tsx   sends signed-out people to /login, wrong roles to /app
│  │  ├─ LoginDialog.tsx      the sign-in / create-account pop-up
│  │  ├─ RolePickerDialog.tsx the "pick a role" pop-up
│  │  ├─ POS.tsx              checkout: basket, member, points, payment
│  │  ├─ ProductMenu.tsx      the product list at checkout, grouped by category
│  │  ├─ ProductDetails.tsx   the pop-up when you open a product at checkout
│  │  ├─ Receipt.tsx          the receipt pop-up, Print and Download PDF
│  │  ├─ Inventory.tsx        products table + Add / Edit / Receive / Write-off forms
│  │  ├─ Orders.tsx           admin: history of every sale and what was in it
│  │  ├─ Loyalty.tsx          members and the points audit log
│  │  ├─ Analytics.tsx        sales numbers and charts (Day/Month/Year with ‹ ›)
│  │  ├─ ProfitLoss.tsx       the profit & loss section of Analytics
│  │  ├─ UserRoles.tsx        give people the cashier/admin role
│  │  ├─ BatchBreakdown.tsx   "B-3: 10 · B-1: 3" stock-per-batch text
│  │  ├─ BarcodeImage.tsx     draws a barcode
│  │  ├─ BrandLogo, PageHeader, LoadingPanel, LoadErrorNotice   small shared pieces
│  │  └─ ui/                  generic controls used everywhere:
│  │     button, input (Input/Textarea), field (label+box+error),
│  │     rich-select (dropdown), category-input (text box with suggestions),
│  │     date-picker (calendar), image-picker (product photo with
│  │     preview and Remove), dialog (pop-up),
│  │     status-text (coloured status words)
```

```
│  │
│  ├─ contexts/               data shared by the whole app
│  │  ├─ AuthContext.tsx      who is signed in, their roles, sign in/out functions
│  │  └─ NotifyContext.tsx    pop-up notices and "Are you sure?" questions
│  │
│  ├─ hooks/useStoreData.ts   loads products, stock, batches, members… for a page
│  │
│  ├─ lib/                    plain logic, no screens
│  │  ├─ supabase.ts          the connection to the database
│  │  ├─ api.ts               EVERY database call goes through here
│  │  ├─ roles.ts             what each role is called, where it goes, how roles are read
│  │  ├─ validation.ts        zod rules for every form
│  │  ├─ loyalty.ts           points maths (₱5 per point)
│  │  ├─ pricing.ts           near-expiry discount maths (15% off)
│  │  ├─ periods.ts           day / month / year browsing for Analytics
│  │  ├─ expiry.ts            date maths in Philippine time
│  │  ├─ categories.ts        category suggestions and grouping
│  │  └─ utils.ts             cn(): joins class names
│  │
│  ├─ types/app.ts            the shapes of our data (Product, Customer, …)
│  └─ ui/roleIcons.tsx        which icon each role uses
│
├─ supabase/migrations/       the database, built up one dated SQL file at a time
├─ supabase/scripts/          one-off data scripts, run by hand (never by db:push):
│                             reset_demo_data.sql = keep accounts, start the store
│                             over with 33 real branded products;
│                             demo_sales.sql = 6 weeks of sample sales through
│                             the real checkout, plus 2 write-offs
└─ backups/                   local data backups (git-ignored: they hold emails)
```

---

4. How the pieces talk to each other

```
Page (e.g. AdminPage)
  │ uses
  ▼
useStoreData()  ──────────►  api.ts  ──────────►  Supabase (database)
  │ gives back products,        every call is          tables + SQL functions,
  │ stock, members… and         one function           protected by row-level
  │ handleCheckout()            (getProducts…)         security
  ▼
Components (POS, Inventory…)
  receive the data as props and call the functions they were given
```

- **Pages** decide *which* components to show and load the data.
- **Components** show data and report what the user did (by calling the functions passed to them as props, e.g. `onCheckout`).
- **`api.ts`** is the only place that talks to the database. If you want to know how anything is saved or loaded, look there.
- **The database** has the final say. Prices, stock and points are always re-checked by SQL functions, so a tampered browser can't change them.

Walk-through: a cashier completes a sale

1. `ProductMenu.tsx`: the cashier scans a barcode; Enter calls `handleScan` in `POS.tsx`, which finds the product and calls `addToCart`.
2. `addToCart` picks which **batch** to sell from (soonest expiry first) and adds a basket line. Near-expiry lines get the 15% markdown (`lib/pricing.ts`) in the preview.
3. The cashier picks a member (name / loyalty ID / phone), chooses Cash or Card and types the amount. If it's short, "Use N points" covers the gap (₱5 per point, `pointsToCover` in `lib/loyalty.ts`); "Pay the whole bill with points" appears when the member has enough.
4. **Complete Transaction** → `handleCheckoutSubmit` checks the basket, the member and the amount (`cashReceivedSchema` / `cardAmountSchema` in `lib/validation.ts`; a card is charged exactly what's due).
5. It calls `onCheckout` → `useStoreData.handleCheckout` → `api.processCheckout` → database function **`process_pos_checkout`**.
6. In the database, in ONE transaction (all or nothing): insert the order → insert the order lines → take the stock off the products and batches → apply points → write the loyalty log. (Real work: `process_pos_checkout_base`; the wrapper adds receipt details.)
7. The result comes back as the **receipt** (`Receipt.tsx`), and the page reloads stock and points so every number is current.

Walk-through: signing in

1. Landing page **Sign in** → URL becomes `/login` → `LandingPage` opens `LoginDialog`.
2. `LoginDialog` checks the email/password with `loginSchema` and calls `signInWithEmail` from `AuthContext`, which asks Supabase.
3. When Supabase says yes, `AuthContext` sets `user`, loads the person's roles (`lib/roles.ts → getMyRoles`), and the landing page sends them to `/app`. Google sign-in comes back to `/` instead; a note left in `sessionStorage` (`rememberSignInRedirect` / `takeSignInRedirect` in `lib/roles.ts`) tells the landing page to send them on to `/app` as well.
4. `/app` (`RolePickerPage`): a plain customer goes straight into the shop (`/self-serve`); a cashier picks Customer or Cashier; an admin picks Customer, Cashier or Admin (`getMyRoles` gives admins the cashier role too).
5. Every role page is wrapped in `ProtectedRoute` (see `App.tsx`), which bounces anyone without that role.

---

5. The database

For the full diagram (every column, key and relationship) see `ERD.md`.

Main tables:

Table	Holds
`users`	everyone's name, email, phone (a PH mobile number, set in Loyalty & Rewards, used to find members at the POS)
`customer`	loyalty ID and points balance per person
`cashier`, `admin`	one row per person who has that role
`products`	catalog: SKU, barcode, brand, name, category, price, cost (admins only), stock, reorder level, picture, description
`product_batches`	each delivery: batch code, how many came in / are left, expiry date, status
`orders`, `order_items`	every sale and its lines: price charged, plus the normal price, cost and name frozen at the time of sale (admins only)
`loyalty_logs`	every point earned or redeemed (the audit trail)
`spoilage_logs`	every unit removed without being sold (expired / damaged / spoiled), at cost

Main SQL functions (called with `supabase.rpc(...)` from `api.ts`):

Function	Does
`process_pos_checkout`	the whole sale, atomically
`get_product_inventory`	products with sellable stock, expired stock and near-expiry counts
`receive_stock`	adds a batch with an expiry date
`write_off_stock`	removes damaged/spoiled units and logs the loss
`process_expired_batches`	moves expired batches out of stock and logs the loss
`get_admin_sales_analytics`	all Analytics numbers (admin only)
`get_sales_breakdown`	payment mix + top products for any date range (admin only)
`get_profit_loss_series`	orders, gross sales, costs, net income per hour / day / month (admin only)
`get_product_costs`	what each product costs the store (admin only)
`set_user_roles`	gives/takes the cashier and admin roles (admin only)
`set_member_phone`	saves a member's phone number (staff for anyone, others for themselves)
`ensure_my_account`	makes sure a new sign-up has a customer/loyalty row

Who sees cost: signed-in users may read every product column *except* `cost` (Postgres column privileges). Admins get costs through `get_product_costs()`. **Order lines are frozen:** a trigger (`snapshot_order_item_product`) copies the product's price, cost and name onto each new order line, and reports and receipts read those copies.

Security: row-level security means the database itself checks who may read or change each row. Functions marked `security definer` run with extra rights, so each one starts by checking `is_admin()` / `is_staff()` or that a customer is only buying for themselves. The role checks in the React code only decide what to *show*; the database is what actually *enforces* them.

Migrations: each file in `supabase/migrations` is one change, applied in date order. A later file replaces a function from an earlier one, so the newest file that mentions a function is the one in use. Never edit an old migration to change the database. Add a new file and run `npm run db:push`.

---

6. Business rules (where each one lives)

Rule	In the browser (preview)	In the database (final)
1 point per full ₱100 paid	(shown on receipt)	`process_pos_checkout_base`
A point is worth ₱5	`lib/loyalty.ts`	`process_pos_checkout_base`
15% off stock expiring within 7 days	`lib/pricing.ts`	`process_pos_checkout_base`
Sell soonest-expiring batch first	`lib/expiry.ts → sellableBatches`	`consume_product_batches` (a trigger runs it whenever an order line is saved)
Low stock = at or below reorder level	`Inventory.tsx`	—
Every sale needs a member	`POS.tsx`	`process_pos_checkout_base`

If you change a rule, change **both** columns.

---

7. Input validation (three layers)

1. **The box itself**: `maxLength`, `type="number"`, etc. stop most typos.
2. **zod rules** in `lib/validation.ts`, checked when a form is submitted. Messages appear under the box that's wrong (`fieldErrors()` + `<Field error>`). Regular expressions are used for SKU (`SKU_PATTERN`) and category (`CATEGORY_PATTERN`) text.
3. **The database**: check constraints (e.g. price ≥ 0) and the checks inside each SQL function. Even if the first two were skipped, bad data is refused.

---

8. Look and feel

- All colours are named in `src/index.css` (`brand`, `ink`, `canvas`, `subtle`, `sun`, `warning`, `danger` …). Components use those names, never raw hex codes, so changing a colour means changing one line.
- Clickable/typeable things are **pills** (`rounded-full`); containers have small corners (`rounded-lg` = 8px).
- Statuses are **plain coloured text** (`ui/status-text.tsx`), never boxed.
- Dropdowns are always `RichSelect`, never the browser's own `<select>`.
- Dates are always `DatePicker` (`ui/date-picker.tsx`), never `<input type="date">`. Text boxes switch off the browser's own suggestion list (`ui/input.tsx`).

9. React ideas you'll see everywhere

Idea	Meaning	Example
Component	A function that returns what to show	`function Receipt(...) { return <div>… }`
Props	Inputs passed to a component	`<Receipt receipt={r} onClose={…} />`
`useState`	A value the component remembers; changing it redraws the screen	`const [cart, setCart] = useState([])`
`useEffect`	Run something *after* drawing, e.g. load data, listen for keys	loading analytics when the tab opens
`useMemo`	Remember a calculated value until its inputs change	category suggestions
`useRef`	Remember something WITHOUT redrawing (or point at a real element)	the double-submit guard in POS
Context	Share data with every component without passing props down	`useAuth()`, `useNotify()`
Custom hook	A reusable function that uses other hooks	`useStoreData()`
`async` / `await`	Wait for the database to answer	`await api.getProducts()`

10. Suggested reading order

1. `src/main.tsx` → `src/App.tsx` (how the app starts and which URL shows what)
2. `src/types/app.ts` (the shapes of the data)
3. `src/lib/supabase.ts` → `src/lib/api.ts` (how data is loaded and saved)
4. `src/contexts/AuthContext.tsx` → `src/lib/roles.ts` → `src/components/ProtectedRoute.tsx`
5. `src/hooks/useStoreData.ts` → `src/pages/AdminPage.tsx`
6. `src/components/POS.tsx` with `ProductMenu.tsx`, `ProductDetails.tsx`, `Receipt.tsx`, and `src/lib/loyalty.ts`, `pricing.ts`, `expiry.ts`
7. `src/components/Inventory.tsx` with `src/lib/validation.ts`
8. `src/components/Analytics.tsx` → `ProfitLoss.tsx`
9. The newest files in `supabase/migrations/` (checkout, analytics)
10. `src/index.css` and `src/components/ui/*` (the look)